

TABLE 10.2 *Decoding table for the (7, 4) Hamming code defined in Table 10.1*

<i>Syndrome</i>	<i>Error Pattern</i>
0 0 0	0 0 0 0 0 0 0
1 0 0	1 0 0 0 0 0 0
0 1 0	0 1 0 0 0 0 0
0 0 1	0 0 1 0 0 0 0
1 1 0	0 0 0 1 0 0 0
0 1 1	0 0 0 0 1 0 0
1 1 1	0 0 0 0 0 1 0
1 0 1	0 0 0 0 0 0 1

[1100010] with an error in the third bit. Using Equation (10.19), the syndrome is calculated to be

$$\begin{aligned}
 s &= [1100010] \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix} \\
 &= [0 \ 0 \ 1]
 \end{aligned}$$

From Table 10.2 the corresponding coset leader (i.e., error pattern with the highest probability of occurrence) is found to be [0010000], indicating correctly that the third bit of the received vector is erroneous. Thus, adding this error pattern to the received vector, in accordance with Equation (10.26), yields the correct code vector actually sent. $\blacktriangleleft$

DUAL CODE

Given a linear block code, we may define its *dual* as follows. Taking the transpose of both sides of Equation (10.15), we have

$$GH^T = 0$$

where H^T is the transpose of the parity-check matrix of the code, and 0 is a new zero matrix. This equation suggests that every (n, k) linear block code with generator matrix G and parity-check matrix H has a *dual code* with parameters $(n, n - k)$, generator matrix H and parity-check matrix G .

10.4 Cyclic Codes

Cyclic codes form a subclass of linear block codes. Indeed, many of the important linear block codes discovered to date are either cyclic codes or closely related to cyclic codes. An

advantage of cyclic codes over most other types of codes is that they are easy to encode. Furthermore, cyclic codes possess a well-defined mathematical structure, which has led to the development of very efficient decoding schemes for them.

A binary code is said to be a *cyclic code* if it exhibits two fundamental properties:

1. *Linearity property*: The sum of any two code words in the code is also a code word.
2. *Cyclic property*: Any cyclic shift of a code word in the code is also a code word.

Property 1 restates the fact that a cyclic code is a linear block code (i.e., it can be described as a parity-check code). To restate Property 2 in mathematical terms, let the n -tuple $(c_0, c_1, \dots, c_{n-1})$ denote a code word of an (n, k) linear block code. The code is a cyclic code if the n -tuples

$$\begin{aligned} &(c_{n-1}, c_0, \dots, c_{n-2}), \\ &(c_{n-2}, c_{n-1}, \dots, c_{n-3}), \\ &\vdots \\ &(c_1, c_2, \dots, c_{n-1}, c_0) \end{aligned}$$

are all code words in the code.

To develop the algebraic properties of cyclic codes, we use the elements $c_0, c_1, \dots, c_{n-1}$ of a code word to define the *code polynomial*

$$c(X) = c_0 + c_1X + c_2X^2 + \dots + c_{n-1}X^{n-1} \quad (10.27)$$

where X is an indeterminate. Naturally, for binary codes, the coefficients are 1s and 0s. Each power of X in the polynomial $c(X)$ represents a one-bit *shift* in time. Hence, multiplication of the polynomial $c(X)$ by X may be viewed as a shift to the right. The key question is: How do we make such a shift *cyclic*? The answer to this question is addressed next.

Let the code polynomial $c(X)$ be multiplied by X^i , yielding

$$\begin{aligned} X^i c(X) &= X^i(c_0 + c_1X + \dots + c_{n-i-1}X^{n-i-1} + c_{n-i}X^{n-i} \\ &\quad + \dots + c_{n-1}X^{n-1}) \\ &= c_0X^i + c_1X^{i+1} + \dots + c_{n-i-1}X^{n-1} + c_{n-i}X^n \\ &\quad + \dots + c_{n-1}X^{n+i-1} \\ &= c_{n-i}X^n + \dots + c_{n-1}X^{n+i-1} + c_0X^i + c_1X^{i+1} \\ &\quad + \dots + c_{n-i-1}X^{n-1} \end{aligned} \quad (10.28)$$

where, in the last line, we have merely rearranged terms. Recognizing, for example, that $c_{n-i} + c_{n-i} = 0$ in modulo-2 addition, we may manipulate the first i terms of Equation (10.28) as follows:

$$\begin{aligned} X^i c(X) &= c_{n-i} + \dots + c_{n-1}X^{i-1} + c_0X^i + c_1X^{i+1} + \dots + c_{n-i-1}X^{n-1} \\ &\quad + c_{n-i}(X^n + 1) + \dots + c_{n-1}X^{i-1}(X^n + 1) \end{aligned} \quad (10.29)$$

Next, we introduce the following definitions:

$$\begin{aligned} c^{(i)}(X) &= c_{n-i} + \dots + c_{n-1}X^{i-1} + c_0X^i + c_1X^{i+1} \\ &\quad + \dots + c_{n-i-1}X^{n-1} \end{aligned} \quad (10.30)$$

$$q(X) = c_{n-i} + c_{n-i+1}X + \dots + c_{n-1}X^{i-1} \quad (10.31)$$

Accordingly, Equation (10.29) is reformulated in the compact form

$$X^i c(X) = q(X)(X^n + 1) + c^{(i)}(X) \quad (10.32)$$

The polynomial $c^{(i)}(X)$ is recognized as the code polynomial of the code word $(c_{n-i}, \dots, c_{n-1}, c_0, c_1, \dots, c_{n-i-1})$ obtained by applying i cyclic shifts to the code word $(c_0, c_1, \dots, c_{n-i-1}, c_{n-i}, \dots, c_{n-1})$. Moreover, from Equation (10.32) we readily see that $c^{(i)}(X)$ is the remainder that results from dividing $X^i c(X)$ by $(X^n + 1)$. We may thus formally state the cyclic property in polynomial notation as follows: *If $c(X)$ is a code polynomial, then the polynomial*

$$c^{(i)}(X) = X^i c(X) \bmod (X^n + 1) \quad (10.33)$$

is also a code polynomial for any cyclic shift i ; the term $\bmod$ is the abbreviation for modulo. The special form of polynomial multiplication described in Equation (10.33) is referred to as *multiplication modulo $X^n + 1$* . In effect, the multiplication is subject to the constraint $X^n = 1$, the application of which restores the polynomial $X^i c(X)$ to order $n - 1$ for all $i < n$. (Note that in modulo-2 arithmetic, $X^n + 1$ has the same value as $X^n - 1$.)

■ GENERATOR POLYNOMIAL

The polynomial $X^n + 1$ and its factors play a major role in the generation of cyclic codes. Let $g(X)$ be a polynomial of degree $n - k$ that is a factor of $X^n + 1$; as such, $g(X)$ is *the polynomial of least degree in the code*. In general, $g(X)$ may be expanded as follows:

$$g(X) = 1 + \sum_{i=1}^{n-k-1} g_i X^i + X^{n-k} \quad (10.34)$$

where the coefficient g_i is equal to 0 or 1. According to this expansion, the polynomial $g(X)$ has two terms with coefficient 1 separated by $n - k - 1$ terms. The polynomial $g(X)$ is called the *generator polynomial* of a cyclic code. A cyclic code is uniquely determined by the generator polynomial $g(X)$ in that each code polynomial in the code can be expressed in the form of a polynomial product as follows:

$$c(X) = a(X)g(X) \quad (10.35)$$

where $a(X)$ is a polynomial in X with degree $k - 1$. The $c(X)$ so formed satisfies the condition of Equation (10.33) since $g(X)$ is a factor of $X^n + 1$.

Suppose we are given the generator polynomial $g(X)$ and the requirement is to encode the message sequence $(m_0, m_1, \dots, m_{k-1})$ into an (n, k) *systematic* cyclic code. That is, the message bits are transmitted in unaltered form, as shown by the following structure for a code word (see Figure 10.4):

$$\underbrace{(b_0, b_1, \dots, b_{n-k-1})}_{n-k \text{ parity bits}}, \underbrace{(m_0, m_1, \dots, m_{k-1})}_{k \text{ message bits}}$$

Let the *message polynomial* be defined by

$$m(X) = m_0 + m_1 X + \dots + m_{k-1} X^{k-1} \quad (10.36)$$

and let

$$b(X) = b_0 + b_1 X + \dots + b_{n-k-1} X^{n-k-1} \quad (10.37)$$

According to Equation (10.1), we want the code polynomial to be in the form

$$c(X) = b(X) + X^{n-k}m(X) \quad (10.38)$$

Hence, the use of Equations (10.35) and (10.38) yields

$$a(X)g(X) = b(X) + X^{n-k}m(X)$$

Equivalently, in light of modulo-2 addition, we may write

$$\frac{X^{n-k}m(X)}{g(X)} = a(X) + \frac{b(X)}{g(X)} \quad (10.39)$$

Equation (10.39) states that the polynomial $b(X)$ is the *remainder* left over after dividing $X^{n-k}m(X)$ by $g(X)$.

We may now summarize the steps involved in the encoding procedure for an (n, k) cyclic code assured of a systematic structure. Specifically, we proceed as follows:

1. Multiply the message polynomial $m(X)$ by X^{n-k} .
2. Divide $X^{n-k}m(X)$ by the generator polynomial $g(X)$, obtaining the remainder $b(X)$.
3. Add $b(X)$ to $X^{n-k}m(X)$, obtaining the code polynomial $c(X)$.

■ PARITY-CHECK POLYNOMIAL

An (n, k) cyclic code is uniquely specified by its generator polynomial $g(X)$ of order $(n - k)$. Such a code is also uniquely specified by another polynomial of degree k , which is called the *parity-check polynomial*, defined by

$$h(X) = 1 + \sum_{i=1}^{k-1} h_i X^i + X^k \quad (10.40)$$

where the coefficients h_i are 0 or 1. The parity-check polynomial $h(X)$ has a form similar to the generator polynomial in that there are two terms with coefficient 1, but separated by $k - 1$ terms.

The generator polynomial $g(X)$ is equivalent to the generator matrix G as a description of the code. Correspondingly, the parity-check polynomial, denoted by $h(X)$, is an equivalent representation of the parity-check matrix H . We thus find that the matrix relation $HG^T = 0$ presented in Equation (10.15) for linear block codes corresponds to the relationship

$$g(X)h(X) \bmod (X^n + 1) = 0 \quad (10.41)$$

Accordingly, we may state that *the generator polynomial $g(X)$ and the parity-check polynomial $h(X)$ are factors of the polynomial $X^n + 1$, as shown by*

$$g(X)h(X) = X^n + 1 \quad (10.42)$$

This property provides the basis for selecting the generator or parity-check polynomial of a cyclic code. In particular, we may state that if $g(X)$ is a polynomial of degree $(n - k)$ and it is also a factor of $X^n + 1$, then $g(X)$ is the generator polynomial of an (n, k) cyclic code. Equivalently, we may state that if $h(X)$ is a polynomial of degree k and it is also a factor of $X^n + 1$, then $h(X)$ is the parity-check polynomial of an (n, k) cyclic code.

A final comment is in order. Any factor of $X^n + 1$ with degree $(n - k)$, the number of parity bits, can be used as a generator polynomial. For large values of n , the polynomial $X^n + 1$ may have many factors of degree $n - k$. Some of these polynomial factors generate

good cyclic codes, whereas some of them generate bad cyclic codes. The issue of how to select generator polynomials that produce good cyclic codes is very difficult to resolve. Indeed, coding theorists have expended much effort in the search for good cyclic codes.

■ GENERATOR AND PARITY-CHECK MATRICES

Given the generator polynomial $g(X)$ of an (n, k) cyclic code, we may construct the generator matrix G of the code by noting that the k polynomials $g(X), Xg(X), \dots, X^{k-1}g(X)$ span the code. Hence, the n -tuples corresponding to these polynomials may be used as rows of the k -by- n generator matrix G .

However, the construction of the parity-check matrix H of the cyclic code from the parity-check polynomial $h(X)$ requires special attention, as described here. Multiplying Equation (10.42) by $a(x)$ and then using Equation (10.35), we obtain

$$c(X)b(X) = a(X) + X^n a(X) \quad (10.43)$$

The polynomials $c(X)$ and $b(X)$ are themselves defined by Equations (10.27) and (10.40), respectively, which means that their product on the left-hand side of Equation (10.43) contains terms with powers extending up to $n + k - 1$. On the other hand, the polynomial $a(X)$ has degree $k - 1$ or less, the implication of which is that the powers of $X^k, X^{k+1}, \dots, X^{n-1}$ do *not* appear in the polynomial on the right-hand side of Equation (10.43). Thus, setting the coefficients of $X^k, X^{k+1}, \dots, X^{n-1}$ in the expansion of the product polynomial $c(X)b(X)$ equal to zero, we obtain the following set of $n - k$ equations:

$$\sum_{i=j}^{j+k} c_i b_{k+j-i} = 0 \quad \text{for } 0 \leq j \leq n - k - 1 \quad (10.44)$$

Comparing Equation (10.44) with the corresponding relation of Equation (10.16), we may make the following important observation: The coefficients of the parity-check polynomial $b(X)$ involved in the polynomial multiplication described in Equation (10.44) are arranged in *reversed* order with respect to the coefficients of the parity-check matrix H involved in forming the inner product of vectors described in Equation (10.16). This observation suggests that we define the *reciprocal of the parity-check polynomial* as follows:

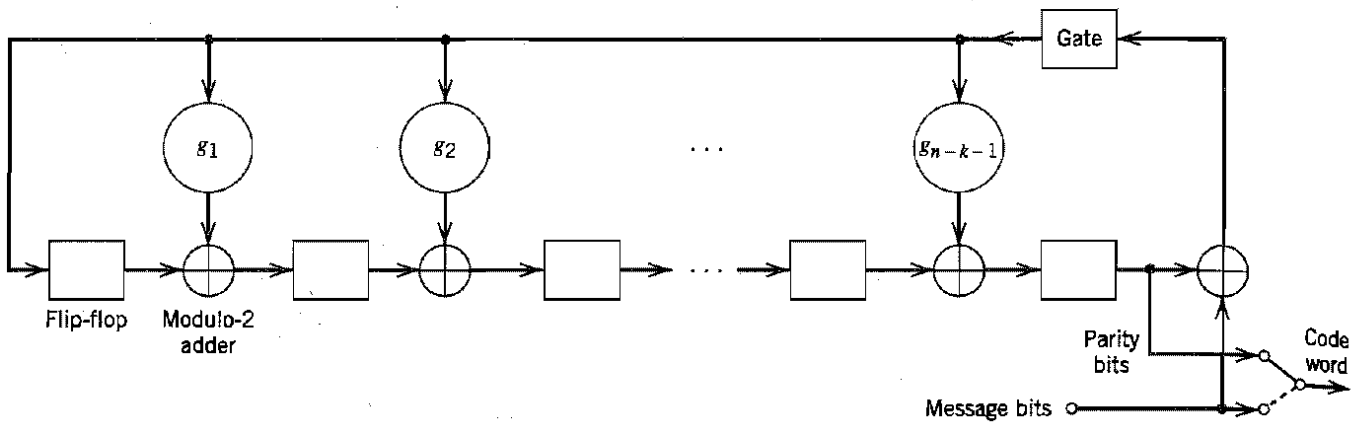
$$\begin{aligned} X^k b(X^{-1}) &= X^k \left(1 + \sum_{i=1}^{k-1} b_i X^{-i} + X^{-k} \right) \\ &= 1 + \sum_{i=1}^{k-1} b_{k-i} X^i + X^k \end{aligned} \quad (10.45)$$

which is also a factor of $X^n + 1$. The n -tuples pertaining to the $(n - k)$ polynomials $X^k b(X^{-1}), X^{k+1} b(X^{-1}), \dots, X^{n-1} b(X^{-1})$ may now be used in rows of the $(n - k)$ -by- n parity-check matrix H .

In general, the generator matrix G and the parity-check matrix H constructed in the manner described here are not in their systematic forms. They can be put into their systematic forms by performing simple operations on their respective rows, as illustrated in Example 10.3.

■ ENCODER FOR CYCLIC CODES

Earlier we showed that the encoding procedure for an (n, k) cyclic code in systematic form involves three steps: (1) multiplication of the message polynomial $m(X)$ by X^{n-k} , (2) di-

FIGURE 10.8 Encoder for an (n, k) cyclic code.

vision of $X^{n-k}m(X)$ by the generator polynomial $g(X)$ to obtain the remainder $b(X)$, and (3) addition of $b(X)$ to $X^{n-k}m(X)$ to form the desired code polynomial. These three steps can be implemented by means of the encoder shown in Figure 10.8, consisting of a *linear feedback shift register* with $(n - k)$ stages.

The boxes in Figure 10.8 represent *flip-flops*, or *unit-delay elements*. The flip-flop is a device that resides in one of two possible states denoted by 0 and 1. An *external clock* (not shown in Figure 10.8) controls the operation of all the flip-flops. Every time the clock ticks, the contents of the flip-flops (initially set to the state 0) are shifted out in the direction of the arrows. In addition to the flip-flops, the encoder of Figure 10.8 includes a second set of logic elements, namely, *adders*, which compute the modulo-2 sums of their respective inputs. Finally, the *multipliers* multiply their respective inputs by the associated coefficients. In particular, if the coefficient $g_i = 1$, the multiplier is just a direct “connection.” If, on the other hand, the coefficient $g_i = 0$, the multiplier is “no connection.”

The operation of the encoder shown in Figure 10.8 proceeds as follows:

1. The gate is switched on. Hence, the k message bits are shifted into the channel. As soon as the k message bits have entered the shift register, the resulting $(n - k)$ bits in the register form the parity bits [recall that the parity bits are the same as the coefficients of the remainder $b(X)$].
2. The gate is switched off, thereby breaking the feedback connections.
3. The contents of the shift register are read out into the channel.

■ CALCULATION OF THE SYNDROME

Suppose the code word $(c_0, c_1, \dots, c_{n-1})$ is transmitted over a noisy channel, resulting in the received word $(r_0, r_1, \dots, r_{n-1})$. From Section 10.3, we recall that the first step in the decoding of a linear block code is to calculate the syndrome for the received word. If the syndrome is zero, there are no transmission errors in the received word. If, on the other hand, the syndrome is nonzero, the received word contains transmission errors that require correction.

In the case of a cyclic code in systematic form, the syndrome can be calculated easily. Let the received word be represented by a polynomial of degree $n - 1$ or less, as shown by

$$r(X) = r_0 + r_1X + \dots + r_{n-1}X^{n-1} \quad (10.46)$$

Let $q(X)$ denote the quotient and $s(X)$ denote the remainder, which are the results of dividing $r(X)$ by the generator polynomial $g(X)$. We may therefore express $r(X)$ as follows:

$$r(X) = q(X)g(X) + s(X) \quad (10.47)$$

The remainder $s(X)$ is a polynomial of degree $n - k - 1$ or less, which is the result of interest. It is called the *syndrome polynomial* because its coefficients make up the $(n - k)$ -by-1 syndrome s .

Figure 10.9 shows a *syndrome calculator* that is identical to the encoder of Figure 10.8 except for the fact that the received bits are fed into the $(n - k)$ stages of the feedback shift register from the left. As soon as all the received bits have been shifted into the shift register, its contents define the syndrome s .

The syndrome polynomial $s(X)$ has the following useful properties that follow from the definition given in Equation (10.47).

1. The syndrome of a received word polynomial is also the syndrome of the corresponding error polynomial.

Given that a cyclic code with polynomial $c(X)$ is sent over a noisy channel, the received word polynomial is defined by

$$r(X) = c(X) + e(X) \quad (10.48)$$

where $e(X)$ is the *error polynomial*. Equivalently, we may write

$$e(X) = r(X) + c(X) \quad (10.49)$$

Hence, substituting Equations (10.35) and (10.47) into (10.49), we get

$$e(X) = u(X)g(X) + s(X) \quad (10.50)$$

where the quotient is $u(X) = a(X) + q(X)$. Equation (10.50) shows that $s(X)$ is also the syndrome of the error polynomial $e(X)$. The implication of this property is that when the syndrome polynomial $s(X)$ is nonzero, the presence of transmission errors in the received word is detected.

2. Let $s(X)$ be the syndrome of a received word polynomial $r(X)$. Then, the syndrome of $Xr(X)$, a cyclic shift of $r(X)$, is $Xs(X)$.

Applying a cyclic shift to both sides of Equation (10.47), we get

$$Xr(X) = Xq(X)g(X) + Xs(X) \quad (10.51)$$

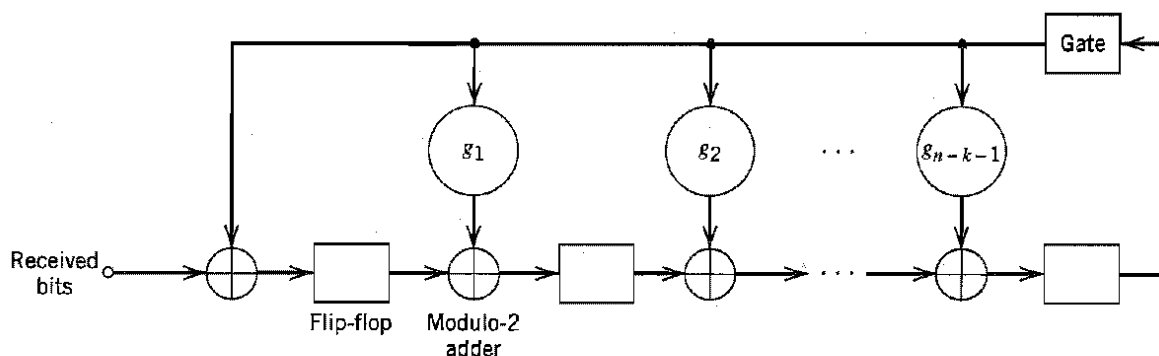


FIGURE 10.9 Syndrome calculator for (n, k) cyclic code.

from which we readily see that $Xs(X)$ is the remainder of the division of $Xr(X)$ by $g(X)$. Hence, the syndrome of $Xr(X)$ is $Xs(X)$ as stated. We may generalize this result by stating that if $s(X)$ is the syndrome of $r(X)$, then $X^i s(X)$ is the syndrome of $X^i r(X)$.

3. The syndrome polynomial $s(X)$ is identical to the error polynomial $e(X)$, assuming that the errors are confined to the $(n - k)$ parity-check bits of the received word polynomial $r(X)$.

The assumption made here is another way of saying that the degree of the error polynomial $e(X)$ is less than or equal to $(n - k - 1)$. Since the generator polynomial $g(X)$ is of degree $(n - k)$, by definition, it follows that Equation (10.50) can only be satisfied if the quotient $u(X)$ is zero. In other words, the error polynomial $e(X)$ and the syndrome polynomial $s(X)$ are one and the same. The implication of Property 3 is that, under the aforementioned conditions, error correction can be accomplished simply by adding the syndrome polynomial $s(X)$ to the received word polynomial $r(X)$.

► EXAMPLE 10.3 Hamming Codes Revisited

To illustrate the issues relating to the polynomial representation of cyclic codes, we consider the generation of a $(7, 4)$ cyclic code. With the block length $n = 7$, we start by factorizing $X^7 + 1$ into three *irreducible polynomials*:

$$X^7 + 1 = (1 + X)(1 + X^2 + X^3)(1 + X + X^3)$$

By an “irreducible polynomial” we mean a polynomial that cannot be factored using only polynomials with coefficients from the binary field. An irreducible polynomial of degree m is said to be *primitive* if the smallest positive integer n for which the polynomial divides $X^n + 1$ is $n = 2^m - 1$. For the example at hand, the two polynomials $(1 + X^2 + X^3)$ and $(1 + X + X^3)$ are primitive. Let us take

$$g(X) = 1 + X + X^3$$

as the generator polynomial, whose degree equals the number of parity bits. This means that the parity-check polynomial is given by

$$\begin{aligned} h(X) &= (1 + X)(1 + X^2 + X^3) \\ &= 1 + X + X^2 + X^4 \end{aligned}$$

whose degree equals the number of message bits $k = 4$.

Next, we illustrate the procedure for the construction of a code word by using this generator polynomial to encode the message sequence 1001. The corresponding message polynomial is given by

$$m(X) = 1 + X^3$$

Hence, multiplying $m(X)$ by $X^{n-k} = X^3$, we get

$$X^{n-k}m(X) = X^3 + X^6$$

The second step is to divide $X^{n-k}m(X)$ by $g(X)$, the details of which (for the example at hand) are given below:

$$\begin{array}{r} \overline{X^3 + X} \\ X^3 + X + 1 \overline{X^6} \\ \underline{X^6} \\ X^4 \\ \underline{X^4} \\ X^2 + X \end{array}$$

Note that in this long division we have treated subtraction the same as addition, since we are operating in modulo-2 arithmetic. We may thus write

$$\frac{X^3 + X^6}{1 + X + X^3} = X + X^3 + \frac{X + X^2}{1 + X + X^3}$$

That is, the quotient $a(X)$ and remainder $b(X)$ are as follows, respectively:

$$a(X) = X + X^3$$

$$b(X) = X + X^2$$

Hence, from Equation (10.38) we find that the desired code polynomial is

$$\begin{aligned} c(X) &= b(X) + X^{n-k}m(X) \\ &= X + X^2 + X^3 + X^6 \end{aligned}$$

The code word is therefore 0111001. The four right-most bits, 1001, are the specified message bits. The three left-most bits, 011, are the parity-check bits. The code word thus generated is exactly the same as the corresponding one shown in Table 10.1 for a (7, 4) Hamming code.

We may generalize this result by stating that *any cyclic code generated by a primitive polynomial is a Hamming code of minimum distance 3*.

We next show that the generator polynomial $g(X)$ and the parity-check polynomial $h(X)$ uniquely specify the generator matrix G and the parity-check matrix H , respectively.

To construct the 4-by-7 generator matrix G , we start with four polynomials represented by $g(X)$ and three cyclic-shifted versions of it, as shown by

$$\begin{aligned} g(X) &= 1 + X + X^3 \\ Xg(X) &= X + X^2 + X^4 \\ X^2g(X) &= X^2 + X^3 + X^5 \\ X^3g(X) &= X^3 + X^4 + X^6 \end{aligned}$$

The polynomials $g(X)$, $Xg(X)$, $X^2g(X)$, and $X^3g(X)$ represent code polynomials in the (7, 4) Hamming code. If the coefficients of these polynomials are used as the elements of the rows of a 4-by-7 matrix, we get the following generator matrix:

$$G' = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix}$$

Clearly, the generator matrix G' so constructed is not in systematic form. We can put it into a systematic form by adding the first row to the third row, and adding the sum of the first two rows to the fourth row. These manipulations result in the desired generator matrix:

$$G = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

which is exactly the same as that in Example 10.2.

We next show how to construct the 3-by-7 parity-check matrix H from the parity-check polynomial $h(X)$. To do this, we first take the *reciprocal* of $h(X)$, namely, $X^4h(X^{-1})$. For the problem at hand, we form three polynomials represented by $X^4h(X^{-1})$ and two shifted versions of it, as shown by

$$\begin{aligned} X^4h(X^{-1}) &= 1 + X^2 + X^3 + X^4 \\ X^5h(X^{-1}) &= X + X^3 + X^4 + X^5 \\ X^6h(X^{-1}) &= X^2 + X^4 + X^5 + X^6 \end{aligned}$$

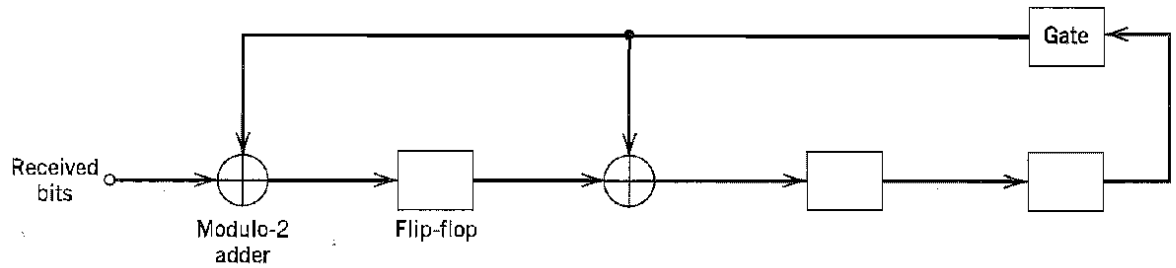


FIGURE 10.11 Syndrome calculator for the (7, 4) cyclic code generated by the polynomial $g(X) = 1 + X + X^3$.

► EXAMPLE 10.4 Maximal-Length Codes

For any positive integer $m \geq 3$, there exists a *maximal-length code* with the following parameters:

$$\begin{aligned} \text{Block length:} & \quad n = 2^m - 1 \\ \text{Number of message bits:} & \quad k = m \\ \text{Minimum distance:} & \quad d_{\min} = 2^{m-1} \end{aligned}$$

Maximal-length codes are generated by polynomials of the form

$$g(X) = \frac{1 + X^n}{b(X)} \quad (10.52)$$

where $b(X)$ is any primitive polynomial of degree m . Earlier we stated that any cyclic code generated by a primitive polynomial is a Hamming code of minimum distance 3 (see Example 10.3). It follows therefore that maximal-length codes are the *dual* of Hamming codes.

The polynomial $b(X)$ defines the feedback connections of the encoder. The generator polynomial $g(X)$ defines one period of the maximal-length code, assuming that the encoder is in the initial state 00...01. To illustrate these points, consider the example of a (7, 3) maximal-length code, which is the dual of the (7, 4) Hamming code described in Example 10.3. Thus, choosing

$$b(X) = 1 + X + X^3$$

we find that the generator polynomial of the (7, 3) maximal-length code is

$$g(X) = 1 + X + X^2 + X^4$$

TABLE 10.4 Contents of the syndrome calculator in Figure 10.11 for the received word 0110001

Shift	Input Bit	Contents of Shift Register
		0 0 0 (initial state)
1	1	1 0 0
2	0	0 1 0
3	0	0 0 1
4	0	1 1 0
5	1	1 1 1
6	1	0 0 1
7	0	1 1 0

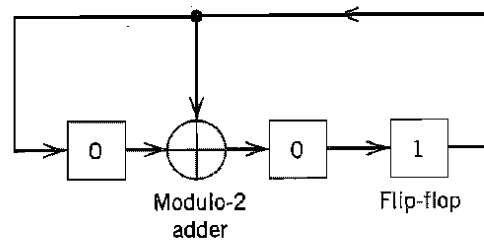


FIGURE 10.12 Encoder for the (7, 3) maximal-length code; the initial state of the encoder is shown in the figure.

Figure 10.12 shows the encoder for the (7, 3) maximal-length code, the feedback connections of which are exactly the same as those shown in Figure 8.2 in Chapter 8. The period of the code is $n = 7$. Thus, assuming that the encoder is in the initial state 001, as indicated in Figure 10.12, we find the output sequence is described by

$$\underbrace{1 \ 0 \ 0}_{\text{initial state}} \quad \underbrace{1 \ 1 \ 1 \ 0 \ 1 \ 0 \ 0}_{g(X) = 1 + X + X^2 + X^4}$$

This result may be readily validated by cycling through the encoder of Figure 10.12.

Note that if we were to choose the other primitive polynomial

$$h(X) = 1 + X^2 + X^3$$

for the (7, 3) maximal-length code, we would simply get the “image” of the code described above, and the output sequence would be “reversed” in time. ◀

■ OTHER CYCLIC CODES

We conclude the discussion of cyclic codes by presenting the characteristics of three other important classes of cyclic codes.

Cyclic Redundancy Check Codes

Cyclic codes are extremely well-suited for *error detection*. We make this statement for two reasons. First, they can be designed to detect many combinations of likely errors. Second, the implementation of both encoding and error-detecting circuits is practical. It is for these reasons that many of the error-detecting codes used in practice are of the cyclic-code type. A cyclic code used for error-detection is referred to as *cyclic redundancy check (CRC) code*.

We define an *error burst* of length B in an n -bit received word as a contiguous sequence of B bits in which the first and last bits or any number of intermediate bits are received in error. Binary (n, k) CRC codes are capable of detecting the following error patterns:

1. All error bursts of length $n - k$ or less.
2. A fraction of error bursts of length equal to $n - k + 1$; the fraction equals $1 - 2^{-(n-k-1)}$.
3. A fraction of error bursts of length greater than $n - k + 1$; the fraction equals $1 - 2^{-(n-k-1)}$.
4. All combinations of $d_{\min} - 1$ (or fewer) errors.
5. All error patterns with an odd number of errors if the generator polynomial $g(X)$ for the code has an even number of nonzero coefficients.

TABLE 10.5 CRC codes

Code	Generator Polynomial, $g(X)$	$n - k$
CRC-12 code	$1 + X + X^2 + X^3 + X^{11} + X^{12}$	12
CRC-16 code (USA)	$1 + X^2 + X^{15} + X^{16}$	16
CRC-ITU code	$1 + X^5 + X^{12} + X^{16}$	16

Table 10.5 presents the generator polynomials of three CRC codes that have become international standards. All three codes contain $1 + X$ as a prime factor. The CRC-12 code is used for 6-bit characters, and the other two codes are used for 8-bit characters. CRC codes provide a powerful method of error detection for use in automatic-repeat request (ARQ) strategies discussed in Section 10.1, and digital subscriber lines discussed in Chapter 4.

Bose–Chaudhuri–Hocquenghem (BCH) Codes⁵

One of the most important and powerful classes of linear-block codes are *BCH codes*, which are cyclic codes with a wide variety of parameters. The most common binary BCH codes, known as *primitive BCH codes*, are characterized for any positive integers m (equal to or greater than 3) and t [less than $(2^m - 1)/2$] by the following parameters:

$$\begin{aligned}\text{Block length:} & \quad n = 2^m - 1 \\ \text{Number of message bits:} & \quad k \geq n - mt \\ \text{Minimum distance:} & \quad d_{\min} \geq 2t + 1\end{aligned}$$

Each BCH code is a *t-error correcting code* in that it can detect and correct up to t random errors per code word. The Hamming single-error correcting codes can be described as BCH codes. The BCH codes offer flexibility in the choice of code parameters, namely, block length and code rate. Furthermore, for block lengths of a few hundred bits or less, the BCH codes are among the best known codes of the same block length and code rate.

A detailed treatment of the construction of BCH codes is beyond the scope of our present discussion. To provide a feel for their capability, we present in Table 10.6, the code parameters and generator polynomials for binary block BCH codes of length up to $2^5 - 1$. For example, suppose we wish to construct the generator polynomial for (15, 7)

TABLE 10.6 Binary BCH codes of length up to $2^5 - 1$

n	k	t	Generator Polynomial							
7	4	1						1	011	
15	11	1						10	011	
15	7	2					111	010	001	
15	5	3				10	100	110	111	
31	26	1						100	101	
31	21	2				11	101	101	001	
31	16	3			1	000	111	110	101	111
31	11	5		101	100	010	011	011	010	101
31	6	7	11	001	011	011	110	101	000	100
										111

Notation: n = block length

k = number of message bits

t = maximum number of detectable errors

The high-order coefficients of the generator polynomial $g(X)$ are at the left.